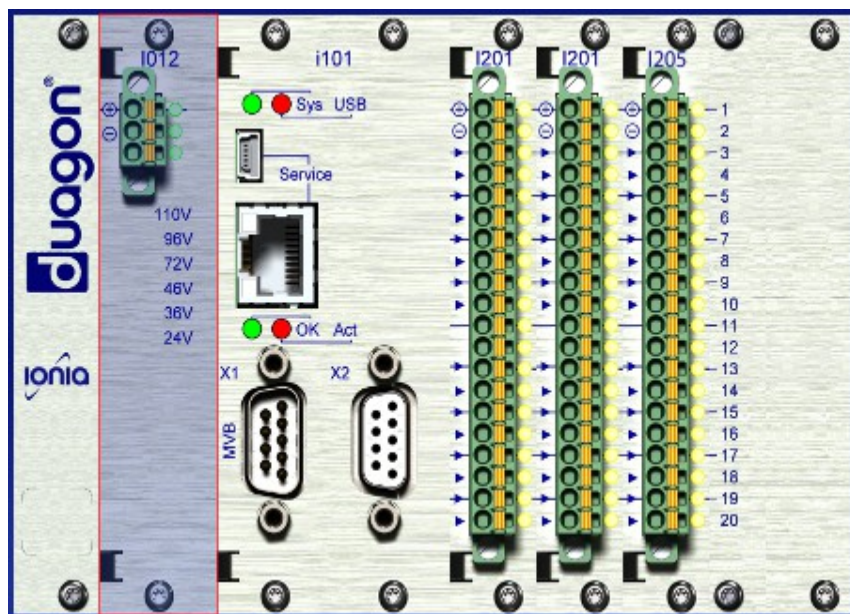




ionia Get Started Documentation

Set-up and Configuration

ionia - a Programmable Logic Controller (PLC) with configurable number of digital/analog input and output channels and fieldbus communication modules. The CPU module can be selected with a fieldbus interface of either MVB, CAN or Ethernet.



duagon AG
Riedstrasse 12
CH-8953 Dietikon
Phone +41 44 743 73 00
Fax +41 44 743 73 15

www.duagon.com

duagon[®]
Moving TCMS Solutions

Duagon Data Sheet Preamble

On having purchased products described in this data sheet, the customer acquires the right to use the products according to its specified purpose and in accordance with all operation, service and maintenance instructions. All other rights to the product, Duagon's intangible assets rights in particular, belong solely to Duagon and may not be deemed to have been assigned along with the sale of the products.

All product properties are fully described in the data sheet under express exclusion of any warranty for other properties. Of decisive relevance is the data sheet valid at the time of the order being placed. Duagon provides a warranty that the product properties are retained during the period of warranty. Evidence that the properties of the product have been retained will be brought, always and exclusively, on Duagon premises by means of a test construction pursuant to the type test.

The customer is obliged to inspect whether the products themselves are suitable for the application intended. In particular, that inspection must include the integration of the products into the intended system configuration and a check on whether the properties as per data sheet can be fulfilled once integrated into the system configuration as planned by the customer. Since the products are not certificated for operation with security applications, the customer must take appropriate measures to ensure that any malfunctions that may occur in a system configuration with other products will be absorbed by supplementary security measures.

The period of warranty for the products is 24 months and it begins on the date the products are shipped from the factory.

The warranty that Duagon assumes for the products will, at Duagon's discretion, be limited either to the repair of or the replacement of the products at the Duagon factory. The warranty solely covers the products or parts thereof which, despite professional handling, have become defective or unusable and which arrive at the Duagon factories for repair or replacement during the period of warranty. The extent of Duagon's warranty is fully set out in this data sheet. Duagon cannot be held liable for consequential damage caused by a defect or for indirect damage or for consequential damage of any kind. Therefore the customer bears all and any costs that occur due to production downtime, for example, or due to the installation or dismantling of products or due to their transportation to Duagon and back.

Duagon's liability and warranty do not obtain if evidence cannot be brought that the products were being operated according to its specified purpose and in accordance with all operation, service and maintenance instructions as issued by Duagon.

These provisions form an integrated part of the product properties. Duagon products cannot be acquired with other or more extensive degrees of warranty and liability on the part of Duagon.

This data sheet is to be evaluated in accordance with **Swiss law**. The court of jurisdiction is the **seat of the vendor**. The applicability of the UN agreement as to international sales of goods (also known as "Viennese Purchasing Convention") is herewith expressly excluded.

duagon AG, Riedstrasse 12, CH-8953 Dietikon, Switzerland

Phone: +41 44 743 73 00, Fax: +41 44 743 73 15, www.duagon.com

Document history:

Rev.	Creator Date	Creator Name	Released Date	Released Name	Comments	Ident-Number
Draft	13-06-03	Bollinger	06.06.2013	Ch.Bollinger	See chapter Document history	d-007724-014426
1	13-08-27	Dent	29.08.2013	J. Dent		d-007724-015008
2	13-09-11	Bollinger	10.02.2014	Ch.Bollinger		d-007724-015105
3	14-05-05	Bollinger	08.05.2014	Ch. Bollinger		d-007724-016712

Table of Contents

1. General Safety Summary.....	4
2. Introduction.....	5
2.1 System Architecture Overview.....	5
3. Hardware Setup.....	6
3.1 Layout.....	6
3.2 Module numbering convention.....	6
3.3 Power Connection.....	7
4. Software Setup.....	8
4.1 MULTIPROG Express Installation.....	8
4.2 DHCP Server.....	9
5. Pre-Installed Sample Applications.....	10
5.1 Selecting a sample application.....	10
5.2 Connection to the Target System.....	10
5.3 Compiling the Application.....	11
5.4 Controlling the Application.....	11
5.5 MVB Application.....	11
5.5.1 i101_MVB_App_001 Multiprog Application.....	13
5.5.2 Duagon MVB Driver.....	13
5.5.3 MVB Port Configuration.....	15
5.6 ETH Application.....	18
5.6.1 i102_UDP_App_001 Multiprog Application.....	18
5.6.2 Duagon Firmware Libraries (for UDP).....	18
5.6.3 Ethernet Application.....	18
5.6.4 ETH Port Configuration.....	19
5.6.5 Using the Ethernet Application.....	19
5.7 WTB Gateway Application: D521_WTB_GW_App_001.....	21
5.7.1 Communication Concept.....	21
5.7.2 MVB Configuration.....	22
5.7.3 Programming Logic.....	23
5.7.4 WTB Configuration.....	25
6. IP configuration.....	27
6.0.1 IP configuration from within an IEC61131 program.....	27
7. References.....	29
8. Document history.....	30
Appendix A: Document Numbering System.....	31

1. General Safety Summary

Review the following safety precautions to avoid injury and prevent damage to this product or those connected to it. To avoid potential hazards, use this product only as specified.

Only qualified personnel should perform installation, maintenance and service procedures

To avoid fire or personal injury

Connect and disconnect properly. This data sheet contains all relevant information for connecting the device.

Power On only with all connections made. All connectors on the device must be connected (unused connections should be covered with a dummy connector).

Ground the product. Ground connection is located on the device.

Observe all terminal ratings. To avoid fire or shock hazard, observe all ratings and markings on the product. Consult this data sheet before making connections to the product.

Avoid exposed circuitry. Do not touch exposed connections and components when power is present.

Do not operate with suspected failures. If you suspect there is a damage to this product, have it inspected by qualified service personnel, or return it to duagon AG.

Safety Terms and Symbols

Terms in this data sheet. These terms may appear in this data sheet.



WARNING. Warning statements identify conditions or practices that could result in injury or loss of life.



CAUTION. Caution statements identify conditions or practices that could result in damage to this product or other property.

Service Safety Summary

Only qualified personnel should perform installation, maintenance and service procedures

Do not service with Power On. Dangerous voltages or currents may exist in this product. Disconnect power, remove battery (if applicable), and disconnect test leads before removing protective panels, soldering, or replacing components.

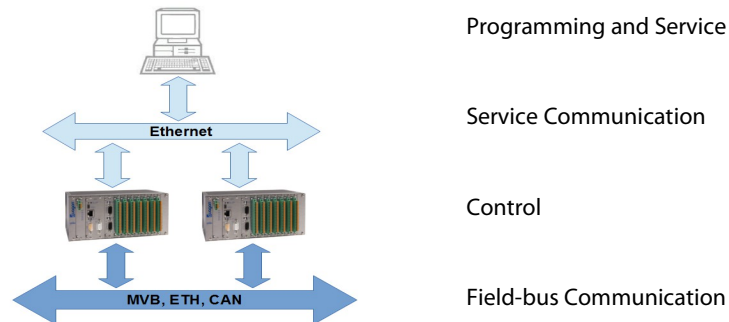
To avoid electric shock, do not touch exposed connections.

2. Introduction

This document describes the necessary steps, used to set up an ionia TCMS system. After following the described steps you will have a full working ionia TCMS device with its IEC61131 programming environment installed.

2.1 System Architecture Overview

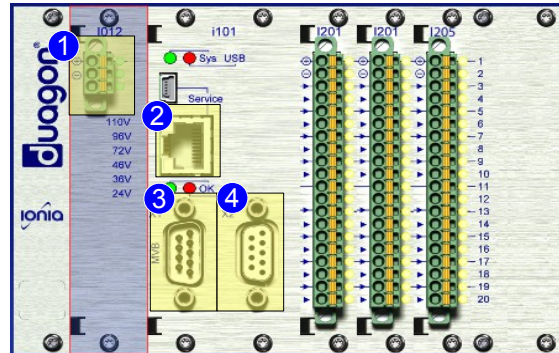
In an ionia TCMS system architecture, the following items can be distinguished:



- Programming and Service
 - A windows based personal computer used as workstation with Multiprog Express programming environment installed.
 - With the Multiprog Express programming environment the TCMS application is programmed in one of the 5 programming languages defined in the IEC61131-3 standard.
 - The configuration of the Field-bus is performed by means of a Field-bus specific configuration tool.
 - In order to access the ionia TCMS system over Ethernet by TCP/IP, a DHCP server installed on this workstation shall provide the desired IP address to the ionia TCMS system.
- Service Communication
 - The service communication is used for download and online debugging of the TCMS application.
- Control
 - A powerful ARM Cortex A8 based CPU executes the TCMS application program.
 - The binary and analogue input channels are read into the TCMS application and processed before being written to the output channels.
 - The same procedure as described for periphery signals is applied to the field-bus signals
- Field-bus communication
 - The field-bus communication is applied according to the actual configuration. The interface to the TCMS application consists of the transmitted and received signals, only.

3. Hardware Set up

3.1 Layout



- ① Power Connector
- ② Ethernet Service Connector
- ③ MVB Fieldbus Connector X1 (male)
- ④ MVB Fieldbus Connector X2 (female)

Note: this example shows the i01, the i02 ETH and i03 CAN have different connectors at ③ and ④.

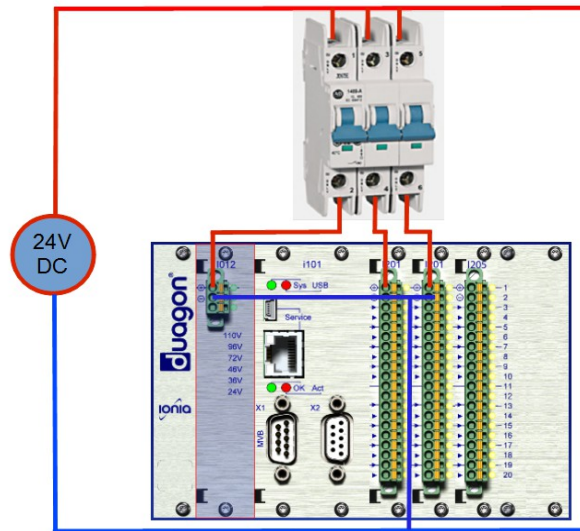
3.2 Module numbering convention

The following module numbering scheme is respected for the ionia modules:

- i012-LV: Module Nr. 00
- i01: Module Nr. 01
- i201: Module Nr. 02
- i202: Module Nr. 03
- i205: Module Nr. 04

3.3 Power Connection

For powering the ionia device, please follow the drawing below.



Note:

1. It is recommended to use independent power circuits to supply the CPU and each i201 rep. i202 card.
2. Binary output card i205 must not be powered by an external power source.

4. Software Set Up

4.1 MULTIPROG Express Installation

In order to guide you with the software installation an installer is provided.

MULTIPROG_Express_d-007271-016712

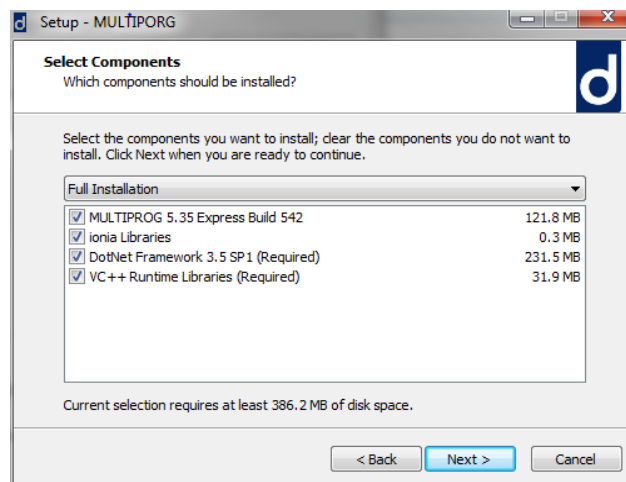
The Multiprog Express installer provides you with:

- Multiprog Express 5.35
- ionia Libraries
- Dot Net Framework 3.5 SP1
- VC++ Runtime Libraries

The first dialog of the installer shows the version information of the installer itself, which serves as identification of the running software.

The next dialog asks for a directory where to install Multiprog Express, if you have no preferences then the default setting is OK.

The third dialog shows all available installation options:



These options can be activated/deactivated by means of setting a flag in the corresponding check box. There are also predefined installation template which you can choose from, by selection a corresponding option in the selection field.

Remark:

If on your computer a component is already installed, than there is no need to install it again, e.g. if on your system Dot Net Framework 3.5. SP1 is already installed then you must not select this option.

If you decide at runtime of the installer, that e.g. Dot Net Framework 3.5. SP1 should not be installed then you can press Cancel the corresponding dialog.

4.2 DHCP Server

A first installation of an ionia TCMS device requires a DHCP server, from which ionia system gets its IP address. This IP address must be known by you. There are various possibilities to master this:

1. If you have access to the IP table of your DHCP server, i.e. you are the network administrator, you can use the DHCP server of your local network.
2. Install a DHCP server on your windows machine which provides the IP address. Various free DHCP servers can be found on the internet.
3. Use a router with a DHCP server service and find the provided IP address by means of your routers Web-server.
4. Install Linux in a virtual machine and activate the DHCP server, e.g. ISC DHCP within Linux. With the following command:
 - `dhcpd eth2 -d -f`
 - eth2 is the interface to your sub-domain.

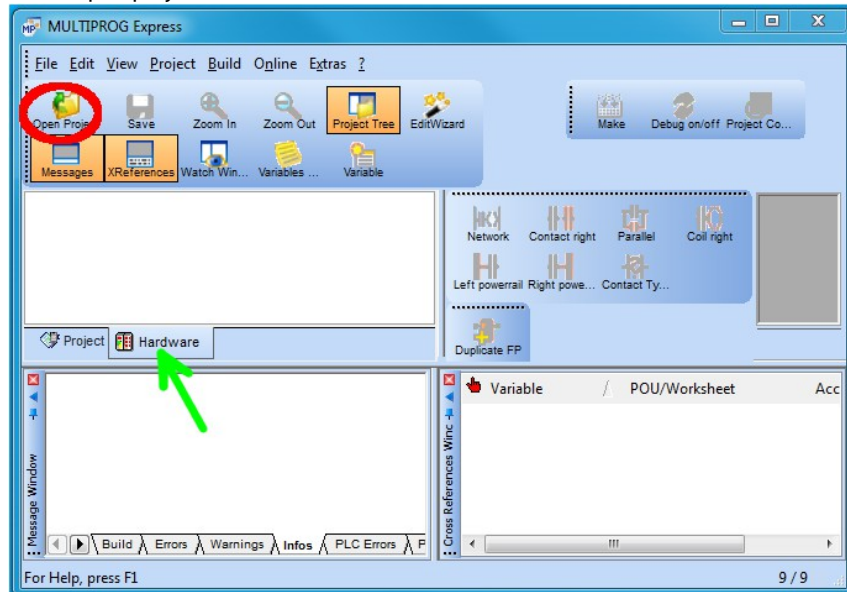
are the log messages sent to the console, thus you can observe which IP address is provided.

Further instructions are given in the man page dhcp.

5. Pre-Installed Sample Applications

5.1 Selecting a sample application

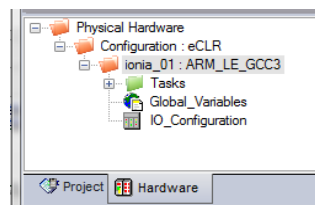
Select open project:



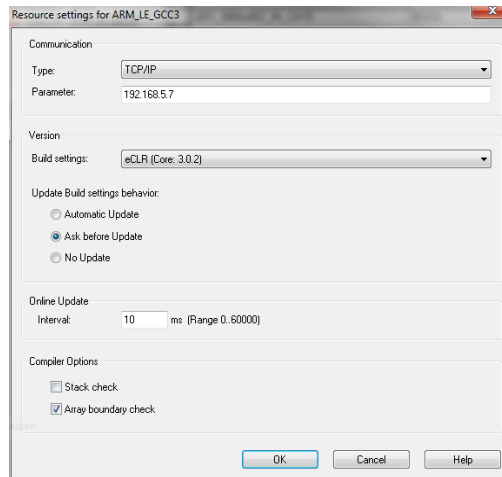
In the Open/Unzip project dialog select an application for example i102_UDP_App_001.mwe or i101_MVB_App_001.mwe as appropriate. When the project has loaded select the hardware tab.

5.2 Connection to the Target System

The configuration of the programming environment and target system is done within the hardware editor by means of a right click on the resource icon and then selecting Settings.

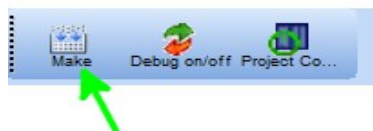


Within the Resource settings dialog the IP address of the ionia system has to be set to the correct value. e.g.:



5.3 Compiling the Application

If any changes are made to an application it will have to be compiled before it can



be downloaded. To do this use the Make button.

5.4 Controlling the Application



Start, Stop and Download an application Program. Click on this button to download a Multiprog application to the ionia system. After a download you will be able to start and stop your application from the corresponding window.



Online debugging with a connected device. Click on this button to start debugging your Multiprog application: signal values are displayed and can be forced to some desired value.

5.5 MVB Application

A MVB system is comprised of at least two nodes. One node, a Class 3 device, act as a slave node whereas the other node, a Class 4 device act, as MVB Bus Administrator and as slave node.

- The administrator is responsible for the bus timing.
- The slave nodes are responsible for the transferred data.

Slave nodes communicate with one other by means of telegrams thus one node may send a telegram to its partner node. Whereas the terminals of a telegram are the source and sink ports located in the traffic store of the communicating nodes.

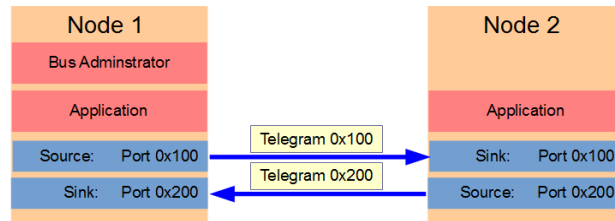


Figure 5.5-1: Sample MVB System

Figure 5.5-1 shows a sample MVB system composed of the MVB nodes. The node ionia_01 has got the source port 0x100 and the receive port 0x200. Node 2 contains the same ports but in reversed direction, i.e. port 0x100 is a sink port whereas port 0x200 is a source port. This enables the MVB system to communicate with one other.

MVB Work-flow

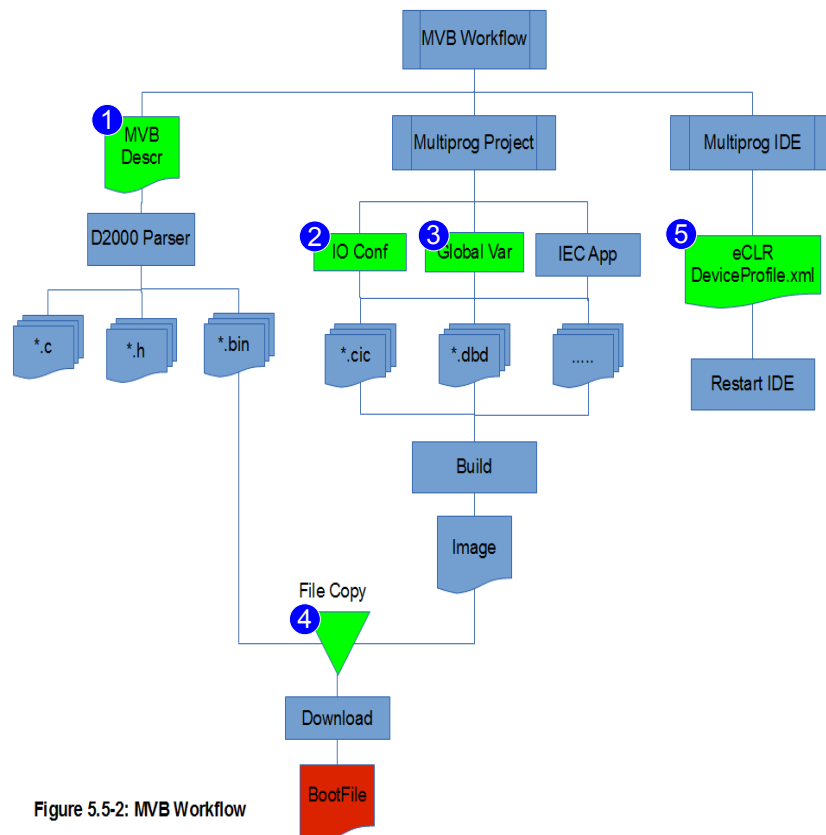


Figure 5.5-2: MVB Workflow

Figure 5.5-2 Shows the work flow used to build an ionia MVB application. Items belonging specific to the building of a MVB application are shown in light green. These are also the process steps where the real engineering work has to be performed, all other processes are processed by the tools themselves.

1

MVB Description File

The MVB description file is the specification the MVB Bus. Among others there are sections for the specification of devices, bus administrator and telegrams. Detailed description of the syntax used in the MVB description file can be found in D2000 MVB Configuration Tool data sheet, d-000341-nnnnnn 1.

Remark:

The Device_Name specified in the MVB Description File must exactly match the resource_name given to the resource in Multiprog.

2

IO Configuration

The IO Configuration specifies the **driver** used to access a certain address range of the global variables. In the case of the i101_DG_MVB_DRV reflect the IO-Configuration the MVB sink and source ports of the device.

3

Global Variables

The individual process variables transmitted by a certain telegram can be accessed by means of an address scheme in the Global_Variables worksheet. Therefore contains the global variables worksheet the data received resp. transmitted over the MVB.

4

File Copy

The D2000 parser generates for each specified device a binary file of the form device_name.bin. The according **Device_Name.bin** has to be copied into the Multiprog project directory:

```
%MPE_PROJDIR%\%PROJE%\C\Configuration\R\resource_name\
```

Remark:

The Device_Name specified in the MVB Description File must exactly match the Resource_Name given to the resource in Multiprog.

5

eCLRDeviceProfile

An entry in the eCLRDeviceProfile.xml is needed, to instruct Multiprog to pack the Device_Name.bin into the download file. After the eCLRDeviceProfile.xml has been modified, a restart of the Multiprog IDE is necessary to load the new configuration.

The eCLRDeviceProfile.xml can be found at the install directory of Multiprog in:

```
%MPE_INSTALLDIR%\PLC\eCLR\ARM_LE_GCC3
```

5.5.1 i101_MVB_GetStarted / i301_MVB_GetStarted

The i101_MVB_GetStarted/i103_MVB_GetStarted application shows the implementation of a MVB node with one source and one sink port with bus administrator capabilities (see figure 5.5-1). Simple process data communication is done by means of reading and writing the first three bytes of the sink port resp. source port. Reading or writing to a port can also be understood as sending and receiving a telegram.

As an example of MVB monitoring, it is shown how to read out the device status word and freshness of the data in the sink port or in other terms the freshness of the receiving telegram, as shown in figure 5.5-1.

Node 1: Is your ionia device with.

ionia is bus administrator and slave node in one device. From the application programmer point of view there is no difference whether the ionia device is a bus administrator or not.

- Device Name: ionia_01
- Resource Name: ionia_02
- MVB Device ID: 0x10

Node 2: d412 MVB Monitor.

Any MVB slave device could be used, but here we use the duagon MVB Monitor d412. The d412 could also be configured as bus administrator but in this example it is used as counterpart to the ionia.

- Device Name: d412
- Resource Name: n.a.
- MVB Device ID: 0x20

It follows a description of implementation details in terms of the above shown process step ① to ⑤.

① MVB Description File: Specifying the MVB Bus Architecture

The i101_MVB_GetStarted/i301_MVB_GetStarted application is comprised of two MVB nodes with two process data telegrams (Figure 5.5-1)

After a default installation of Multiprog is the D2000 project in the following directory located:

- c:\Users\Public\Documents\DG_Tools\D2000\i101_GetStarted

A short description of the content of the D2000 directory with respect to the i101_MVB_GetStarted sample application.

c:\Users\Public\Documents\DG_Tools\D2000>

d2000.exe	- the d2000 executable
d2000.txt	- generic sample mvb description file
d2000_ds.doc	- the d2000 data sheet
d2000_ds.pdf	- the d2000 data sheet as pdf
\---i101_GetStarted	- Root directory of the i101_GetStarted D2000 Project
d2000_log.txt	- Log File of the last parser run
genmbv.bat	-> Dobble click: calls d2000 with all parameter
mvb_ba_0.txt	- Output file of the Bus Administrator configuration
+---out	- All output files from the parser are placed here
d412.bin	- Bin file for the d412 MVB Monitor
d412.c	
d412.h	
dev_addr_list.txt	
ionia_01.bin	- Bin file for the Multiprog Resource ionia_01
ionia_01.c	
ionia_01.h	
md_funct_list.txt	
tel_addr_list.txt	
\---src	

i101_GetStarted.txt -> MVB Description file for the i101_MVB_GetStarted

The important sections of the i101_GetStarted.txt

\$\$DEVICES

The device list contains two devices ionia_01 and d412 with its device ID. More

```

/*****
/* The following section defines the devices list.
*****/
$$DEVICES

$$$DEVICE ionia_01 0x10 BAFULL
$$$MD_PARAMETERS
$$$$$MAX_CALL_NUMBER 5
$$$$$MAX_INST_NUMBER 10
$$$$$DEFAULT_REPLY_TIMEOUT 400
$$$$$MY_CREDIT 7

$$$DEVICE d412 0x20 BAFULL
$$$MD_PARAMETERS
$$$$$MAX_CALL_NUMBER 5
$$$$$MAX_INST_NUMBER 10
$$$$$DEFAULT_REPLY_TIMEOUT 400
$$$$$MY_CREDIT 7

```

devices can be added here (the \$\$\$MD_PARAMETERS can be taken from the example).

\$\$BA

```

/*****
/* The following section defines the Bus Administrator(s).
*****/
$$BA
// List of devices that act as bus administrators.
$$$BA_DEVICE ionia_01
//$$$BA_DEVICE d412

// Parameter definition, valid for all BA devices on this MVB.
$$$BA_PARAMETERS
$$$ACTUALISATION_KEY 0x0000
$$$ST_REPLY_MAX 0
$$$EVENT_POLL_STRATEGY LOW
$$$BASIC_PERIOD 1000
$$$MACROCYCLES_PER_TURN 16
$$$DEVICES_SCAN_STRATEGY ALL

```

The List of bus administrators contains the ionia_01 as bus administrator only, the device d412 is commented out. The \$\$\$BA_PARAMETERS can be taken from the example.

\$\$PD_COMM

```

/*****
/* The following section defines the process data telegrams
/* between intelligent devices.
*****/
$$PD_COMM

$$$PD_TELEGRAM Tgm_256 d412 0x100 B32 32 AUTO
$$$SINK_DEVICE ionia_01

$$$PD_TELEGRAM Tgm_512 ionia_01 0x200 B32 32 AUTO
$$$SINK_DEVICE d412

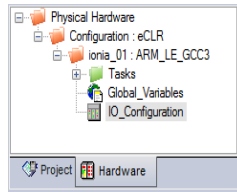
```

\$\$PD_COMM defines the process data telegrams by means of

- Telegram name i.e. Tgm_256
- Source device i.e. d412
- Port number i.e. 0x100
- Port size i.e. B32 (32 Byte)
- Cycle Time i.e. 32 (32 ms)
- Time Slot i.e. AUTO (Configuration done by D2000)
- Sink Device i.e. ionia_01

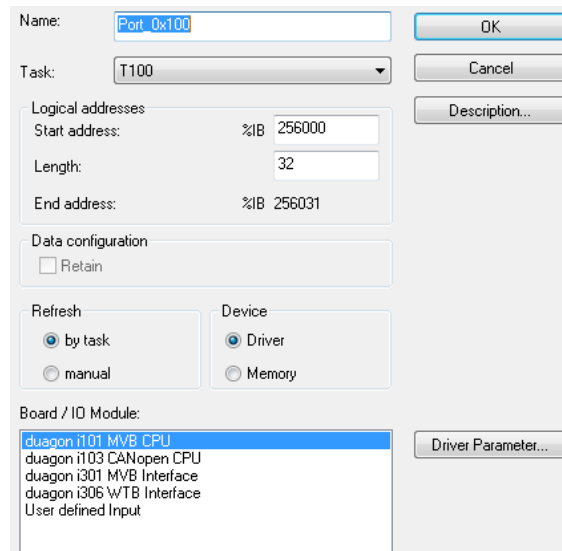
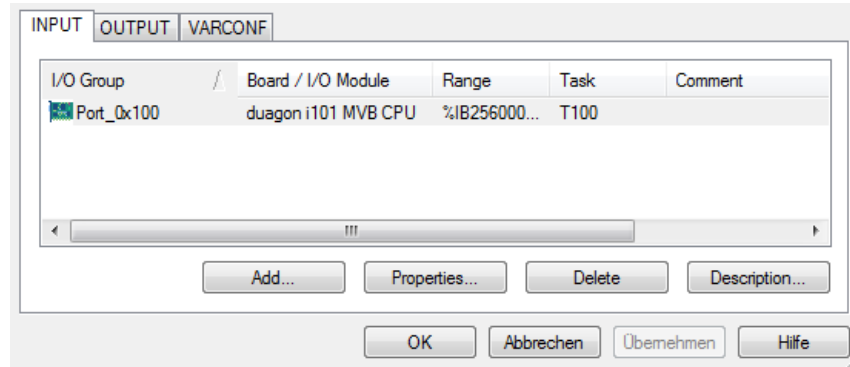
More than one sink device can be added to a telegram. Each sink device receives the same telegram and contains therefore a copy of the process data in its traffic store of the appropriate MVB port.

2 IO Configuration: Selecting the Driver



By means of a double click on the IO_Configuration icon it is possible to load and configure the duagon MVB Driver.

For each MVB port an IO-Group has to be defined. In order to distinguish source and sink port an input resp. an output tab exists in the dialog.



To add a new port i.e. a new IO-Group, the Add button has to be pressed and the form filled in:

Name i.e. Port_0x100 freely selectable

Task i.e. T100

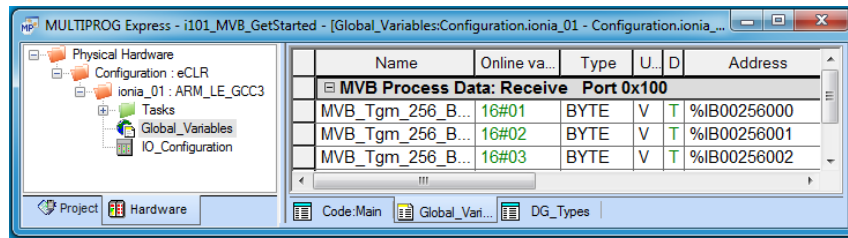
Start address i.e. 256000 := Port Number * 1000

- Length i.e. The port size in byte
- Board IO Mod. i.e. duagon i101 MVB CPU → Driver Selection

Remark:

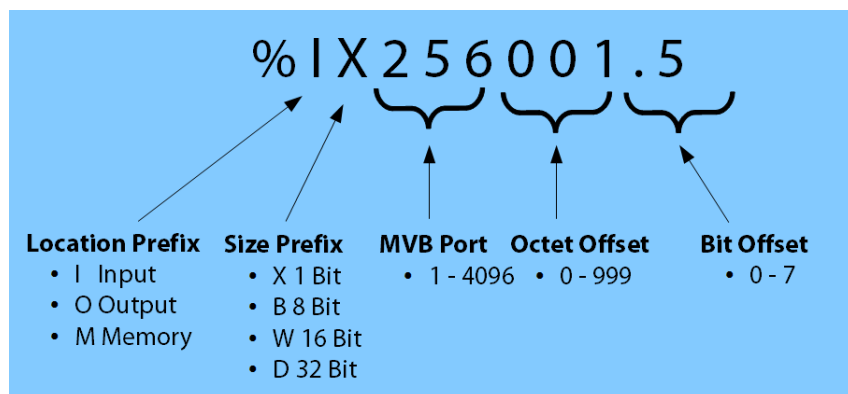
- i101_MVB_GetStarted duagon i101 MVB CPU driver is used
- i301_MVB_GetStarted duagon i301 MVB Interface driver is used

3 Global Variables



The above picture shows the Global_Variables worksheet in debugging mode, thus showing the actual process values in the Online Value column. The variables are accessed by an address, specified in the Address field.

The definition of the address itself follows the IEC-61131 standard with the following syntax:



The above example points to the 6th Bit of the second Byte of Port 0x100.

4 File Copy

The ionia_01.bin file generated with the D2000 parser was copied into to Multiprog Project directory:

Source Directory

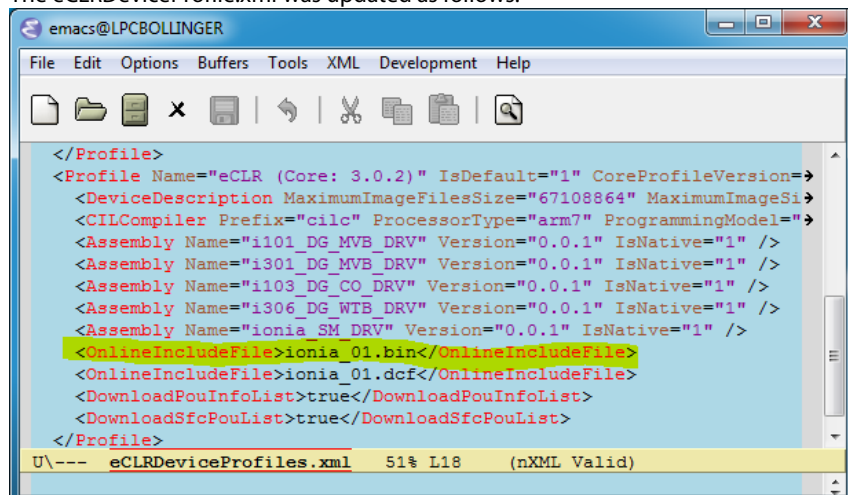
c:\Users\Public\Documents\DG_Tools\D2000\i101_GetStarted\out\ionia_01.bin

Sink Directory

c:\Users\Public\Documents\MULTIPROG\Projects\i101_MVB_GetStarted\C\Configur
ation\R\ionia_01\

5 eCLRDeviceProfile.xml

The eCLRDeviceProfile.xml was updated as follows:



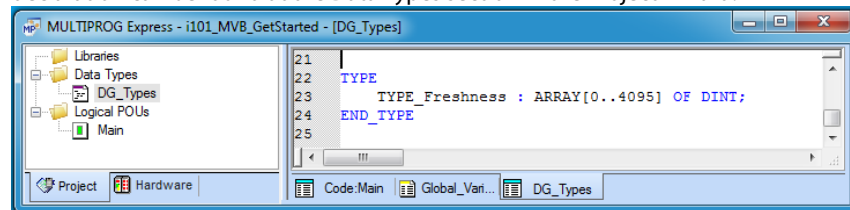
The amendment of the include file has to be done for every <Profile ...> specified.

The eCLRDeviceProfile.xml can be found in the following directory: c:\Program Files (x86)\duagon\MULTIPROG 5.35 Express Build 542\plc\eCLR\

Freshness

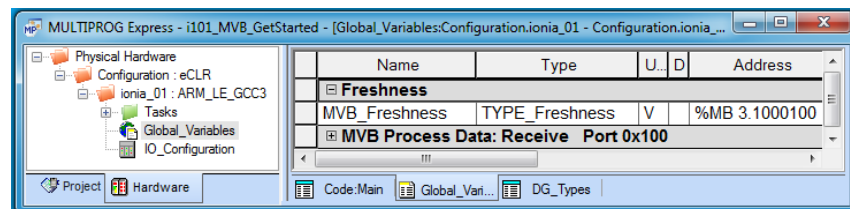
On the ionia the freshness can be read out. The freshness of each configurable port is written into an array.

The data structure of the freshness array has to be declared at application. The declaration can be found at the Data Types section in the Project Wizard.

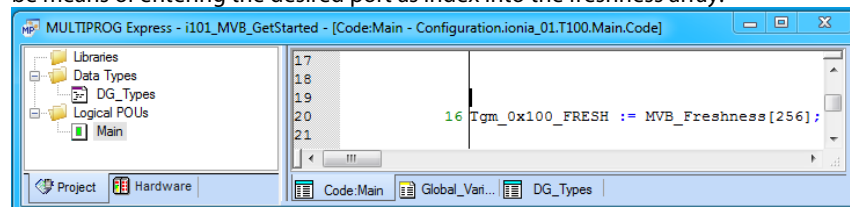


This array can be access by its base address at

- %MB 3.1000100 Freshness array base addressed



After all the declaration and definition is done, the actual freshness can be read out be means of entering the desired port as index into the freshness array.



MVB Driver Initialized

The address of the boolean variable MVB_INITIALIZED is:

- %MX 3. 1000000.0 MVB_INITIALIZED

MVB Device Address

The address of the device ID is:

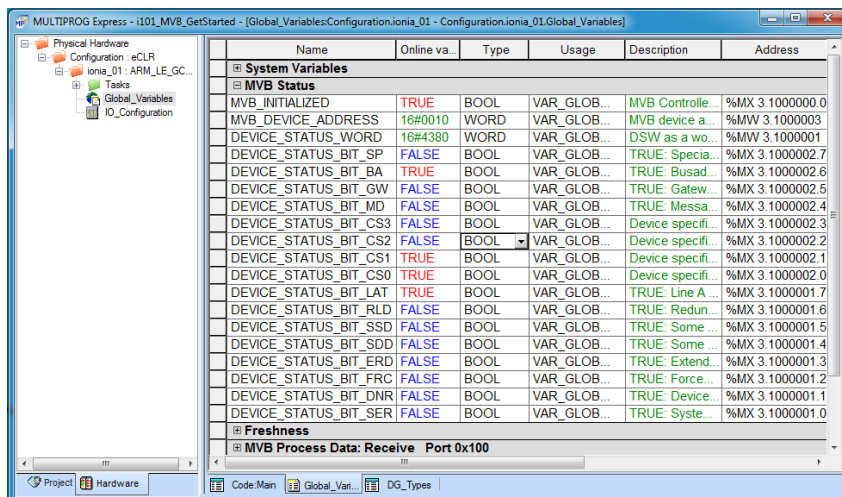
- %MW 3. 1000003 Address Device ID

Device Status Word DSW

The address of the device status word is:

- %MW 3. 1000001 Address of the device status word

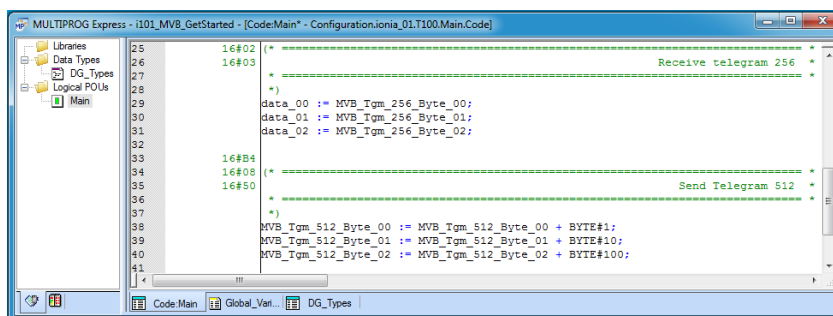
Since every bit of the device status word has its own meaning the DSW can be broken down into more details:



Name	Online va...	Type	Usage	Description	Address
System Variables					
MVB Status					
MVB_INITIALIZED	TRUE	BOOL	VAR_GLOB...	MVB Controlle	%MX 3.1000000.0
MVB_DEVICE_ADDRESS	16#0010	WORD	VAR_GLOB...	MVB device a	%MW 3.1000003
DEVICE_STATUS_WORD	16#4380	WORD	VAR_GLOB...	DSW as a wo	%MW 3.1000001
DEVICE_STATUS_BIT_SP	FALSE	BOOL	VAR_GLOB...	TRUE: Specia	%MX 3.1000002.7
DEVICE_STATUS_BIT_BA	TRUE	BOOL	VAR_GLOB...	TRUE: Busad	%MX 3.1000002.8
DEVICE_STATUS_BIT_GW	FALSE	BOOL	VAR_GLOB...	TRUE: Gatew	%MX 3.1000002.5
DEVICE_STATUS_BIT_MD	FALSE	BOOL	VAR_GLOB...	TRUE: Messa	%MX 3.1000002.4
DEVICE_STATUS_BIT_CS3	FALSE	BOOL	VAR_GLOB...	Device specifi	%MX 3.1000002.3
DEVICE_STATUS_BIT_CS2	FALSE	BOOL	VAR_GLOB...	Device specifi	%MX 3.1000002.2
DEVICE_STATUS_BIT_CS1	TRUE	BOOL	VAR_GLOB...	Device specifi	%MX 3.1000002.1
DEVICE_STATUS_BIT_CS0	TRUE	BOOL	VAR_GLOB...	Device specifi	%MX 3.1000002.0
DEVICE_STATUS_BIT_LAT	TRUE	BOOL	VAR_GLOB...	TRUE: Line A	%MX 3.1000001.7
DEVICE_STATUS_BIT_RLD	FALSE	BOOL	VAR_GLOB...	TRUE: Redun	%MX 3.1000001.6
DEVICE_STATUS_BIT_SSD	FALSE	BOOL	VAR_GLOB...	TRUE: Some	%MX 3.1000001.5
DEVICE_STATUS_BIT_SDD	FALSE	BOOL	VAR_GLOB...	TRUE: Some	%MX 3.1000001.4
DEVICE_STATUS_BIT_ERD	FALSE	BOOL	VAR_GLOB...	TRUE: Extend	%MX 3.1000001.3
DEVICE_STATUS_BIT_FRC	FALSE	BOOL	VAR_GLOB...	TRUE: Force	%MX 3.1000001.2
DEVICE_STATUS_BIT_DNR	FALSE	BOOL	VAR_GLOB...	TRUE: Device	%MX 3.1000001.1
DEVICE_STATUS_BIT_SER	FALSE	BOOL	VAR_GLOB...	TRUE: Syste	%MX 3.1000001.0
Freshness					
MVB Process Data: Receive Port 0x100					

Process Data Communication

The process data communication is applied by means of reading and writing to the global variables defined in step ③:



```

25      16#02 (* ===== Receive telegram 256 ===== *)
26      16#03
27
28      *)
29      data_00 := MVB_Tgm_256.Byte_00;
30      data_01 := MVB_Tgm_256.Byte_01;
31      data_02 := MVB_Tgm_256.Byte_02;
32
33      16#B4
34      16#08 (* ===== Send Telegram 512 ===== *)
35      16#50
36
37      *)
38      MVB_Tgm_512.Byte_00 := MVB_Tgm_512.Byte_00 + BYTE#1;
39      MVB_Tgm_512.Byte_01 := MVB_Tgm_512.Byte_01 + BYTE#10;
40      MVB_Tgm_512.Byte_02 := MVB_Tgm_512.Byte_02 + BYTE#100;
41

```

5.6 CANopen Slave Application

A CANopen system is comprised of at least two nodes, a CANopen Slave i.e. the NMT Slave and a CANopen Master i.e. the NMT Master.

- The NMT Master controls the communication state of the network nodes and does the node monitoring.
- The slave nodes are responsible for its data transfer.

The CANopen Master and CANopen Slave have equal rights from the data communication point of view.

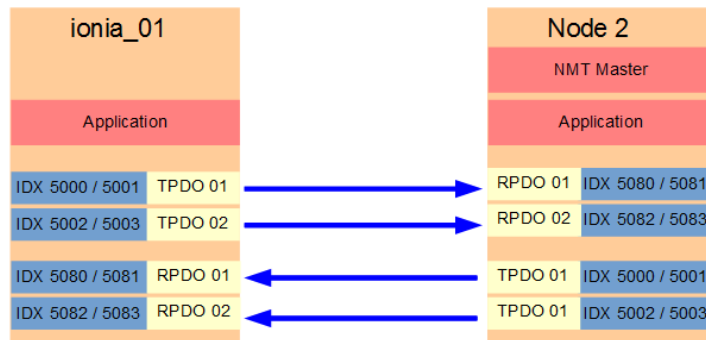


Figure 5.6-1: Sample CANopen System

In the above sample the two nodes communicate by means of exchanging process data objects PDO's. The linking of a Transmit PDO with a corresponding Receive PDO by means of the CobID (CANopen Object ID) is named PDO Linking. Whereby is understood by PDO Mapping which data fields are mapped into the individual PDO, i.e. Object Index 0x5000 and 0x5001 are mapped into TPDO 01.

States and Changes of State

By default a CANopen slave enters the pre-operational state after the boot up procedure. In pre operational state, the nodes may be configured by means of Service Data Objects (SDO).

The NMT Master set the slave node into the operation mode where process data communication (PDO) is possible.

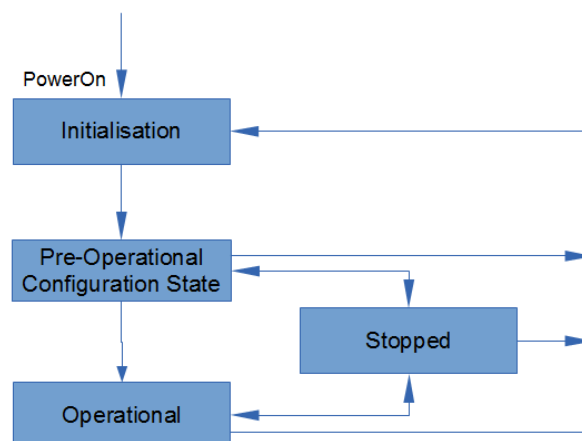


Figure 5.6-1: CANopen Node State

CANopen Work flow

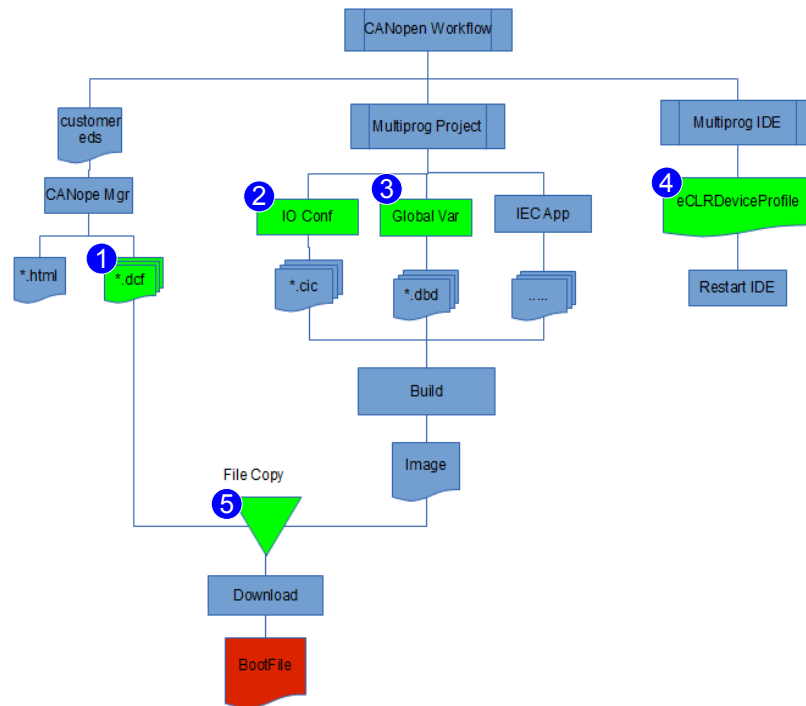


Figure 5.6-2: CANopen Workflow

Figure 5.6-2 Shows the work flow used to build an ionia CANopen application. Items belonging specific to the building of a CANopen application are shown in light green. These are also the process steps where the real engineering work has to be performed, all other processes are processed by the tools themselves.

1 Device Configuration File

The device configuration contains device specific settings such as the Node ID, Baud Rate and PDO Mapping. It is build on behalf of

- CANopen Configuration Manger
- CANopen Electronic Data Sheet.

Remark:

The name of the Device Configuration File must exactly match the resource_name, given to the resource in Multiprog.

2 IO Configuration

The IO Configuration specifies the **driver** used to access a certain address range of the global variables. In case of the i103_DG_CO_DRV driver reflect the IO-Configuration a CANopen objects.

3 Global Variables

The individual process variables transmitted by a certain telegram can be accessed by means of an address scheme in the Global_Variables worksheet. Therefore contains the global variables worksheet the data received resp. transmitted over the can bus

4 File Copy

The CANopen Manger Tool generates for each specified device a text file of the form device_name.dcf. The according **Device_Name.dcf** file has to be copied into the Multiprog project directory:

%MPE_PROJDIR%\%PROJE%\C\Configuration\R\resource_name

Remark:

The name of the Device Configuration File must exactly match the resource_name given to the resource in Multiprog.

5 eCLRDeviceProfile

An entry in the eCLRDeviceProfile.xml is needed, to instruct Multiprog to pack the Device_Name.bin into the download file. After the eCLRDeviceProfile.xml has been modified, a restart of the Multiprog IDE is necessary to load the new configuration.

The eCLRDeviceProfile.xml can be found at the install directory of Multiprog in:

%MPE_INSTALLDIR%\PLC\eCLR\ARM_LE_GCC3

5.6.1 i103_CO_SlaveGetStarted

The i103_CO_SlaveGetStarted application shows the implementation of a CANopen Slave node. Simple process data communication is done by means of reading from RPDO1 and RPDO2 and writing to TPDO1 and TPDO2 (see figure 5.6.-1)

Node 1: Your ionia device

The ionia device is a CANopen Slave and must be set into operational mode by a NMT Master device.

- Device Name: ionia_01
- Resource Name: ionia_01
- CAN Node-ID: 0x32

Node 2: A CAN Monitor

Node 2 is any device which can act as NMT Master. For test reason is often is a CAN Monitor used as counterpart to the ionia device.

- Device Name: dummy master
- Resource Name: n.a.
- MVB Device ID: e.g. 0x64

It follows a description of implementation details in terms of the above shown process step 1 to 5 .

1 Device Configuration File: Specifying the CANopen Architecture

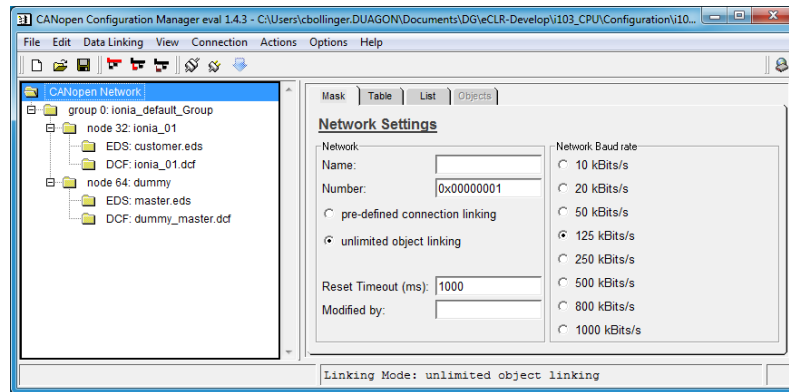
From the communication point of view is the PDO Linking similar to the CANopen architecture since with the PDO linking it is determined which nodes are talking to one other and over which Process Data Objects.

For the i103_CO_SlaveGetStarted application we need at least a dummy master to build a valid dcf file. We use the CANopen Configuration Manager v 1.4.3. from Port to do that job.

After a default installation of Multiprog is the Port project in the following directory located:

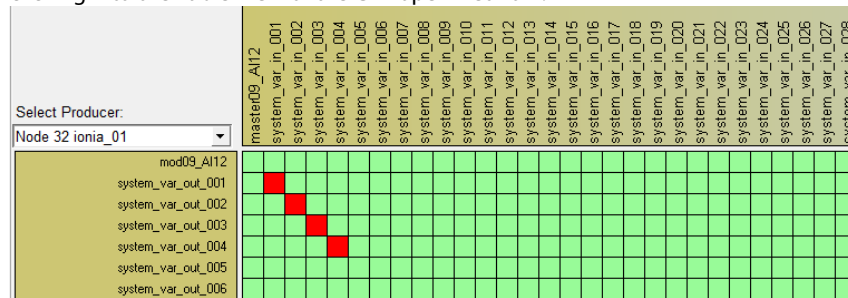
- c:\Users\Public\Documents\DG_Tools\Port\i103_CO_SlaveGetStarted

When opening the i103_CO_GetStarted.net file with the Port CANopen Configuration Manager we get the following overview:



Which shows two nodes, our ionia_01 node and a dummy node used to build a simple CANopen network. The customer.eds reflects the hardware configuration of your ionia device and serves as input file. The master.eds file is the opposite of the customer.eds just to have a counterpart to it.

Once everything is set up properly, the PDO linking can be done by means of clicking into the Table View of the CANopen Network:



In our example we just connected four variables from one node to four variables of the other node and vice versa. All the PDO Linking and PDO Mapping is done by the tool itself, thus we get the following result:

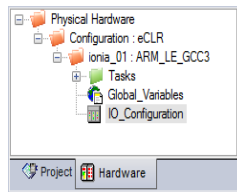
Receive PDOs:					
PDO	COB-ID	Trans Type	Inhibit Time (0.1ms)	Event Timer (ms)	Mappings
1	0x000002C0	0x00 - acyclic	-	-	0x50820020 - system_var_in_003
2	0x000001C0	0x00 - acyclic	-	-	0x50830020 - system_var_in_004
3	0x80000580	0x00 - acyclic	-	-	0x50800020 - system_var_in_001
4	0x80000580	0x00 - acyclic	-	-	0x50810020 - system_var_in_002

Transmit PDOs:					
PDO	COB-ID	Trans Type	Inhibit Time (0.1ms)	Event Timer (ms)	Mappings
1	0x000001A0	0xff - async	0	0	0x50000020 - system_var_out_001
2	0x000002A0	0xff - async	0	0	0x50010020 - system_var_out_002
3	0x80000580	0xff - async	0	0	0x50020020 - system_var_out_003
4	0x80000580	0xff - async	0	0	0x50030020 - system_var_out_004

The object dictionary contains the following CANopen objects from which can be chosen from:

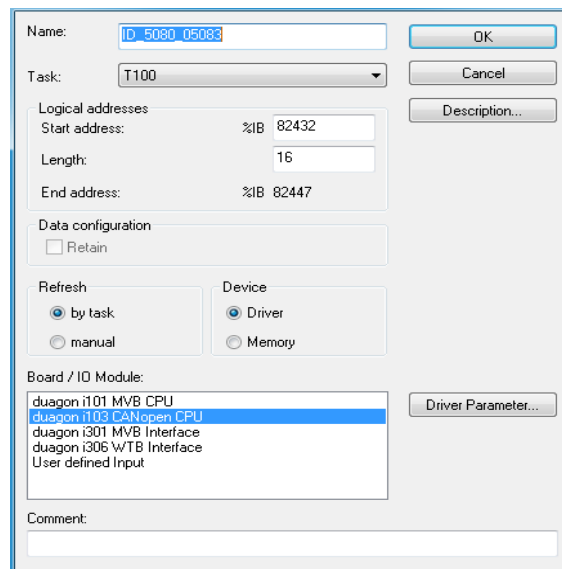
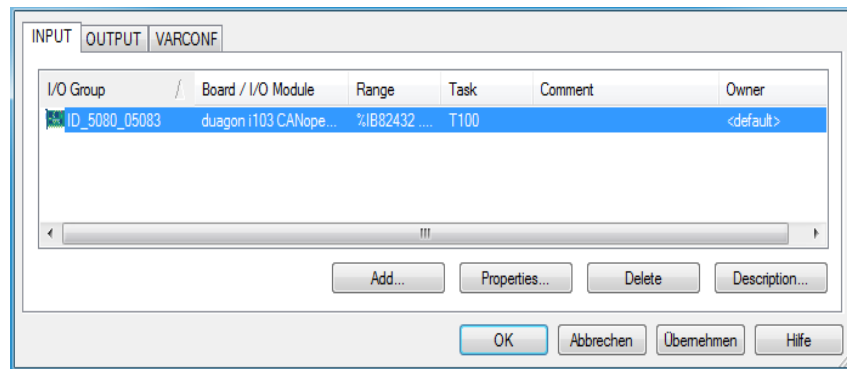
CANopen Index	Number	Data Type	Used for
0x2000 – 0x20FF	256	UNSIGNED8	IO Mapping
0x3000 – 0x30FF	256	UNSIGNED16	IO Mapping
0x4000 – 0x40FF	256	UNSIGNED32	IO Mapping
0x5000 – 0x50FF	256	UNSIGNED32	Mapping of any variables

2 IO Configuration: Selecting the Driver



By means of a double click on the IO_Configuration icon it is possible to load and configure the duagon MVB Driver.

For each MVB port an IO-Group has to be defined. In order to distinguish source and sink port an input resp. an output tab exists in the dialog.



To add a new port i.e. a new IO-Group, the Add button has to be pressed and the form filled in:

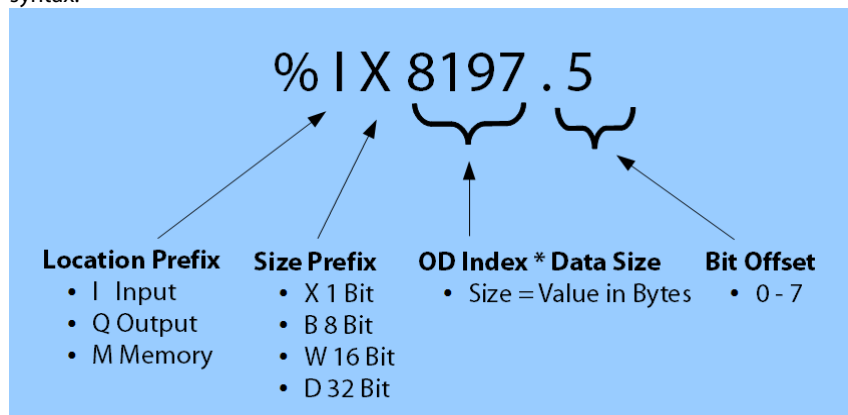
- Name i.e. ID_5080_5083 freely selectable
- Task i.e. T100
- Start address i.e. 82432 := OD Idx * Data Type Size [Byte]
- Length i.e. The size in byte
- Board IO Mod. i.e. duagon i103 CANOpen CPU → Driver Selection

3 Global Variables

Name	Online value	Type	Usage	D...	Address
Send PDO					
system_var_out_001	16#0000040E	DWORD	VA...		%QD81920
system_var_out_002	16#0000288C	DWORD	VA...		%QD81924
system_var_out_003	16#0000040E	DWORD	VA...		%QD81928
system_var_out_004	16#0000288C	DWORD	VA...		%QD81932

The above picture shows the Global_Variables worksheet in debugging mode, thus showing the actual process values in the Online Value column. The variables are accessed by an address, specified in the Address field.

The definition of the address itself follows the IEC-61131 standard with the following syntax:



The above example points to the 5th Bit of OD Index 0x2005.

4 File Copy

The ionia_01.dcf file generated with Port CANopen Configuration Manager was copied into to Multiprog Project directory:

Source Directory

c:\Users\Public\Documents\DG_Tools\D2000\i101_GetStarted\out\ionia_01.bin

Sink Directory

c:\Users\Public\Documents\MULTIPROG\Projects\i103_CO_SlaveGetStarted\C\Confi
guration\R\ionia_01\

5 eCLRDeviceProfile.xml

The eCLRDeviceProfile.xml was updated as follows:

```

</Profile>
<Profile Name="eCLR (Core: 3.0.2)" IsDefault="1" CoreProfileVersion=>
  <DeviceDescription MaximumImageFileSize="67108864" MaximumImageSi>
  <CILCompiler Prefix="cilec" ProcessorType="arm7" ProgrammingModel=>
  <Assembly Name="i101_DG_MVB_DRV" Version="0.0.1" IsNative="1" />
  <Assembly Name="i101_DG_MVB_DRV" Version="0.0.1" IsNative="1" />
  <Assembly Name="i103_DG_CO_DRV" Version="0.0.1" IsNative="1" />
  <Assembly Name="i103_DG_CO_DRV" Version="0.0.1" IsNative="1" />
  <Assembly Name="i106_DG_WTB_DRV" Version="0.0.1" IsNative="1" />
  <Assembly Name="i106_DG_WTB_DRV" Version="0.0.1" IsNative="1" />
  <Assembly Name="ionia_SM_DRV" Version="0.0.1" IsNative="1" />
  <OnlineIncludeFile>ionia_01.bin</OnlineIncludeFile>
  <OnlineIncludeFile>ionia_01.dcf</OnlineIncludeFile>
  <DownloadPouInfoList>true</DownloadPouInfoList>
  <DownloadSfcPouList>true</DownloadSfcPouList>
</Profile>
  
```

U\--- eCLRDeviceProfiles.xml 51% L18 (nXML Valid)

The amendment of the include file has to be done for every <Profile ...> specified.

The eCLRDeviceProfile.xml can be found in the following directory: c:\Program Files (x86)\duagon\MULTIPROG 5.35 Express Build 542\plc\eCLR\

PDO_Transmit_Change

The boolean variable PDO_Transmit_Change enables / disables the immediate sending of PDO's if a contained process variable has changed its value.

PDO_Transmit_Change = TRUE

→ Transmit PDO as soon as a value has changed.

PDO_Transmit_Change = FALSE

→ Transmit PDO according PDO transmission type setting

NMT_State

Shows the nodes NMT State:

- NMT State 0: Initialisation
- NMT State 1: Stopped
- NMT State 2: Operational
- NMT State 3: Pre-Operational
- NMT State 4: Restart Node
- NMT State 5: Restart Communication

DG_State

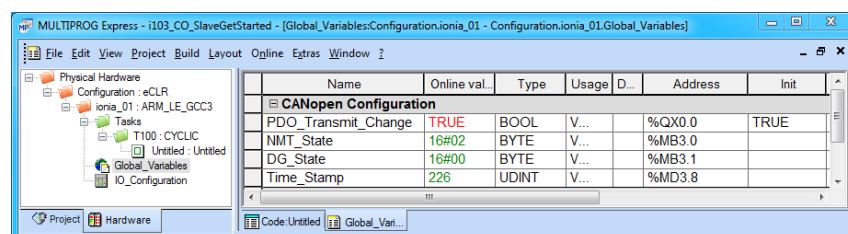
Shows the nodes CAN Controller Status

- DG State 0: OK
- DG State 1: Rx overrun
- DG State 2: Error active
- DG State 3: Error passive
- DG State 4: Bus off
- DG State 5: Bus warning
- DG State 9: Generic error

Time_Stamp

Shows the CANopen timestamp

Overview of the above mentioned CANopen Configuration variables



Name	Online val.	Type	Usage	D...	Address	Init
CANopen Configuration						
PDO_Transmit_Change	TRUE	BOOL	V...		%QX0.0	TRUE
NMT_State	16#02	BYTE	V...		%MB3.0	
DG_State	16#00	BYTE	V...		%MB3.1	
Time_Stamp	226	UDINT	V...		%MD3.8	

5.7 UDP/IP Socket Application

A UDP/IP system is comprised of at least two nodes, which communicate over the ether, therefore we have on each node a sender and a receiver.

One node is your ionia device whereas the other node is just your workstation.

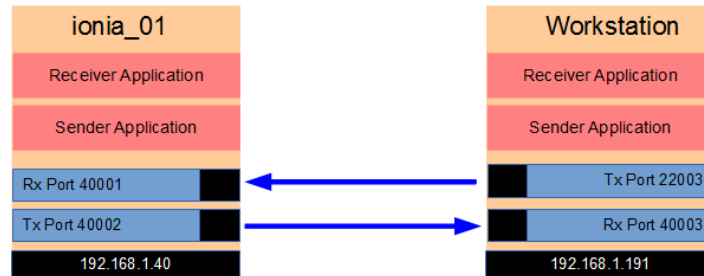


Figure 5.7-1: Sample UDP/IP System

The above two nodes communicate by means of exchanging predefined datagrams in both directions.

UDP Socket Work flow

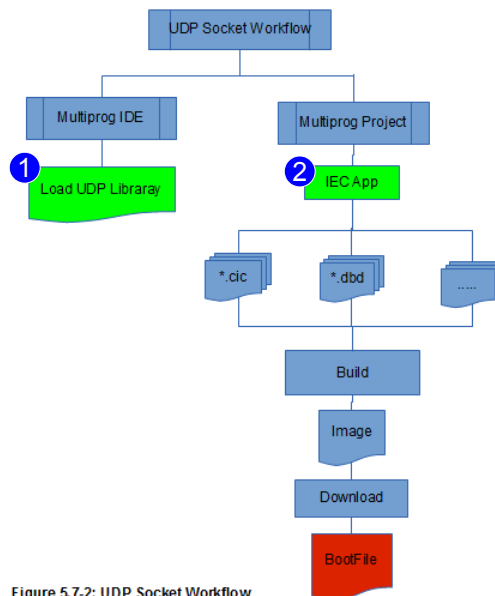


Figure 5.7-2: UDP Socket Workflow

Figure 5.7-2 Shows the work flow used to build an ionia UDP Socket application. Items belonging specific to the building of a UDP Socket application are shown in light green. These are also the process steps where the real engineering work has to be performed, all other processes are processed by the tool itself.

1 Load UDP Library

In order to get access to the Ethernet interface we need to load the corresponding UDP Library. Two UDP libraries exists

- i102_DG_UDP_LIB - Ethernet CPU modules
- i303_DG_UDP_LIB - Ethernet Interface modules

The above mentioned libraries provide function blocks to configure, read and write to the Ethernet interface.

2 IEC Application

Vital to a UDP socket application is the programming task itself, since the

function blocks embedded by the UDP libraries have to be implemented properly (see the next chapter).

5.7.1 i102_UDP_GetStarted / i303_UDP_GetStarted

The i102_UDP_GetStarted rep. the i303_UDP_GetStarted application shows the implementation of a UDP/IP node. Data communication is done by means of sending to resp. reading from appropriate ports (see figure 5.7.-1)

Node 1: Your ionia device

The Ethernet adapter of the ionia device is configured as follows:

- Device Name: ionia_01
- Resource Name: ionia_01
- IP Address: 192.168.1.40
- Receive Port: 40001
- Transmit Port: 40002

Node 2: Workstation

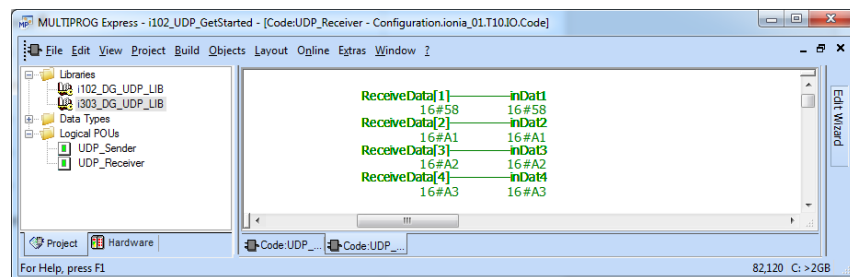
A workstation can be used as counterpart to the ionia device. If Multiprog is running on the same machine a second network interface card (NIC) has to be installed. The configuration for the NIC is as follows:

- Device Name: n.a.
- Resource Name: n.a.
- IP Address: 192.168.1.91
- Receive Port: 40003
- Transmit Port: 22003

It follows a description of implementation details in terms of the above shown process step 1 to 2.

1 Load UDP Libraries

In load one of the following libraries into the Project Editor of Multiprog.



By means of a right mouse-click on the libraries symbol additional libraries can be added.

The default location of the delivered libraries is as follows:

- c:\ProgramData\KW-Software\MULTIPROG Express\5_35_542\plc\FW_LIB\

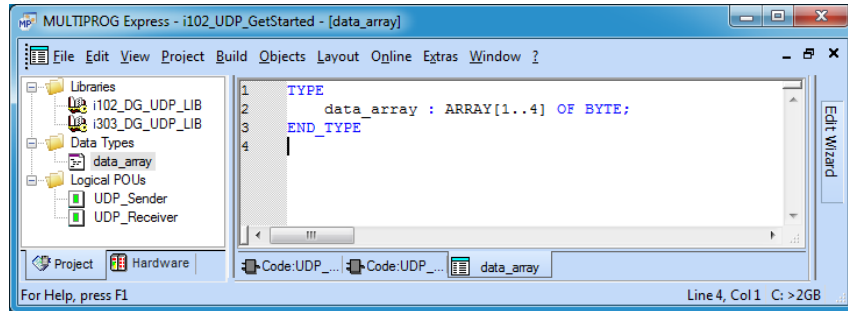
The i102_DG_UDP_LIB and the i303_DG_UDP_LIB contain the same function blocks with one essential difference:

- All function blocks belonging to the Ethernet interface card have got a Module_Nr pin, where the position of the interface module can be determined.

2 Program IEC Application

IMPORTANT: Adjust the Module_Nr according your hardware!

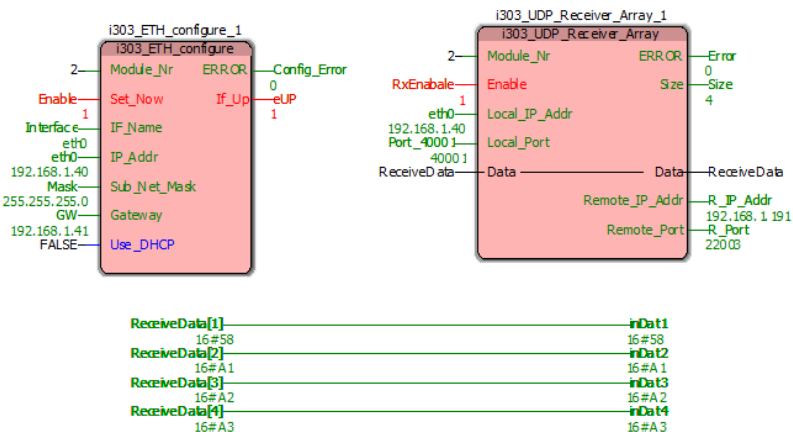
The data structure which is used to send and receive data is defined in the Data_Types branch. An array of 4 Bytes is declared for the input and output stream.



In order to separate the sender and receive logic two independent POU's are applied, the UDP Sender and the UDP Receiver.

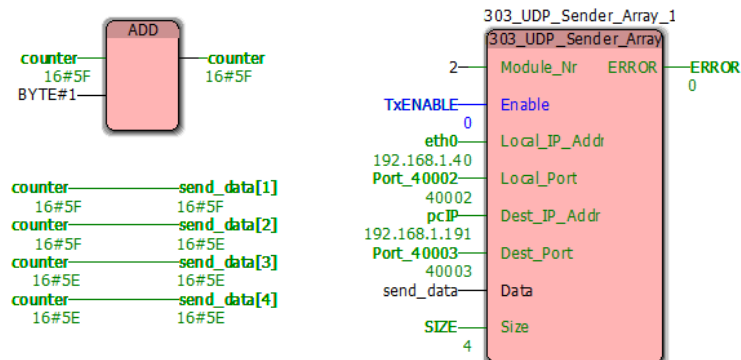
The UDP receiver is built around two function blocks:

- i303_ETH_configure : Configures the network adapter.
- i303_UDP_Receiver_Array : Reads out the data stream



After having received the data stream it is byte wise unpacked.

The UDP Sender consist of a counter and a sender function block. The sender is used to generate test values which are packed into the data stream send_data.



Remark 1:

Please remember before compiling and running this example, the Module_Nr has to be adjusted according your hardware. How to determine the Module_Nr is described in IEC61131 API, d-006935-nnnnnn1.

Remark 2:

The TxEnable and RxEnable variable have to be forced to TRUE before the communication starts.

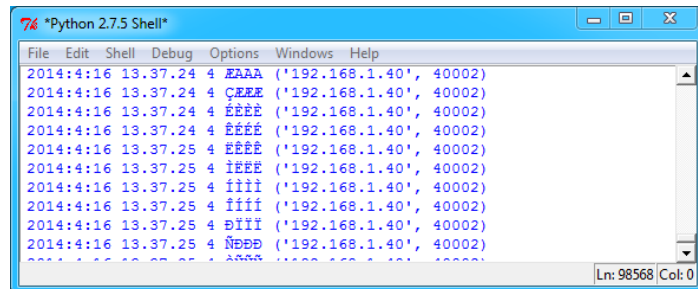
Testing the Application

For test reason two python scripts, which can be run on a workstation, which is configured as mentioned, are supplied. After a MULTIPROG standard installation, can the python script be found at:

1. c:\Users\Public\Documents\DG_Tools\Python\UDP_Recv_4ByteData.py
2. c:\Users\Public\Documents\DG_Tools\Python\UDP_Send_4ByteData.py

The output of the running python scripts are shown below:

Recv_4ByteData.PY

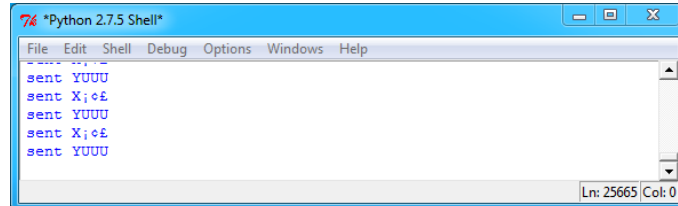


```

File Edit Shell Debug Options Windows Help
2014:4:16 13.37.24 4 EAAA ('192.168.1.40', 40002)
2014:4:16 13.37.24 4 ÇEEE ('192.168.1.40', 40002)
2014:4:16 13.37.24 4 EEEE ('192.168.1.40', 40002)
2014:4:16 13.37.24 4 EEEE ('192.168.1.40', 40002)
2014:4:16 13.37.25 4 EEEE ('192.168.1.40', 40002)
2014:4:16 13.37.25 4 EEEE ('192.168.1.40', 40002)
2014:4:16 13.37.25 4 EEEE ('192.168.1.40', 40002)
2014:4:16 13.37.25 4 EEEE ('192.168.1.40', 40002)
2014:4:16 13.37.25 4 EEEE ('192.168.1.40', 40002)
2014:4:16 13.37.25 4 EEEE ('192.168.1.40', 40002)
2014:4:16 13.37.25 4 EEEE ('192.168.1.40', 40002)
2014:4:16 13.37.25 4 EEEE ('192.168.1.40', 40002)
2014:4:16 13.37.25 4 EEEE ('192.168.1.40', 40002)
2014:4:16 13.37.25 4 EEEE ('192.168.1.40', 40002)
2014:4:16 13.37.25 4 EEEE ('192.168.1.40', 40002)
Ln: 98568 Col: 0

```

Send_4ByteData.PY



```

File Edit Shell Debug Options Windows Help
sent YUUU
sent X;øE
sent YUUU
sent X;øE
sent YUUU
Ln: 25665 Col: 0

```

5.8 Serial Communication

The ionia i305 Serial interface contains two serial ports where each can be configured to conform the following standard:

- RS232
- RS422
- RS485

For our example we use the i305 board in RS232 configuration

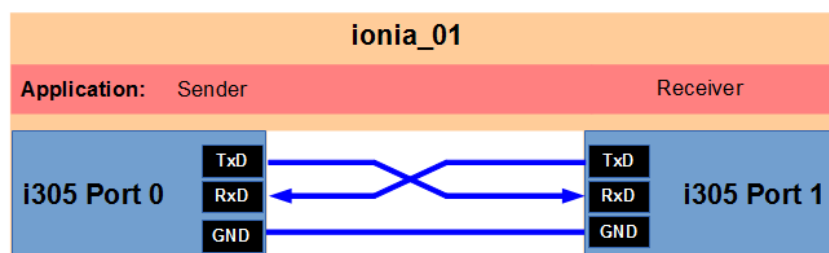


Figure 5.8-1: Sample RS232 Communication

Within this application the ionia controls the two ports from within the same application. Port 0 sends "hello ionia" to port 1.

Serial Port Work flow

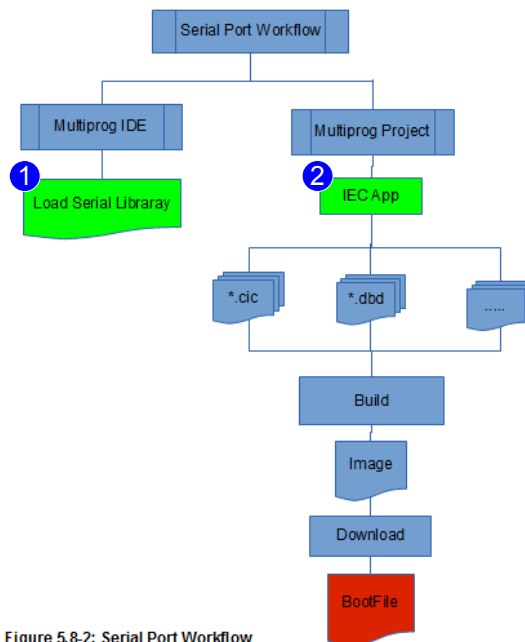


Figure 5.8-2: Serial Port Workflow

Figure 5.7-2 Shows the work flow used to build an ionia serial port application. Items belonging specific to the building of a serial port application are shown in light green. These are also the process steps where the real engineering work has to be performed, all other processes are processed by the tool itself.

1 Load Serial Library

In order to get access to the serial port we need to load the corresponding library:

- i305_DG_SER_LIB

The above mentioned libraries provide function blocks to configure, read and write to the serial port interface.

2 IEC Application

Vital to a serial port application is the programming task itself, since the function blocks embedded by the serial port library have to be implemented properly (see the next chapter).

5.8.1 i305_RS232_GetStarted

IMPORTANT: Adjust the Module_Nr according your hardware!

The i305_RS232_GetStarted application shows the implementation one node with two independent serial ports. The two ports are communication with one other over the serial link. Data communication is done by means of sending to resp. reading from the corresponding port (see figure 5.8.-1)

Configuration of Port 0 and Port 1

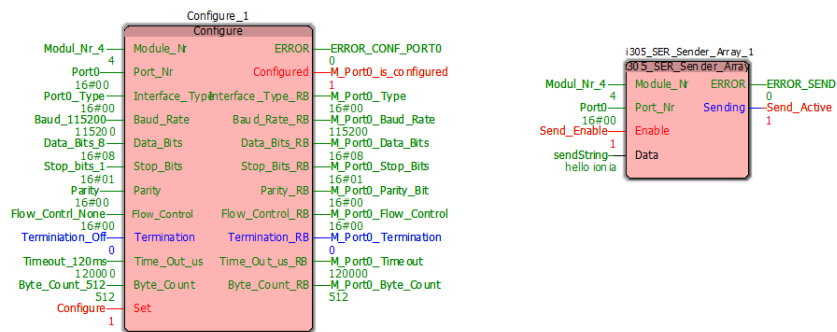
- Interface Type: RS232
- Baud Rate: 115200
- Data Bits: 8
- Stop Bit: 1

- Parity: none
- Flow Control: none

The i305_RS232_GetStarted is comprised of four function blocks only. One function block to configure each serial port and each one block for sending resp. reading data.

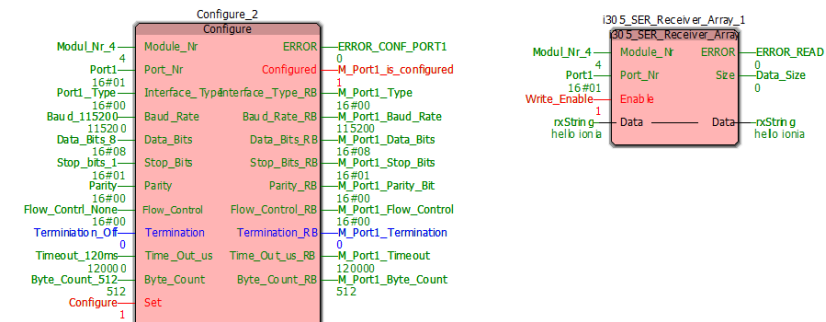
Sender

The function blocks of the sender show the configuration block and the Sender_Array block. The online debugging modes reveals that the configuration was applied as mentioned. The sender block itself reflects that the string "hello ionia" is sent.



Receiver

The function blocks of the receiver show the configuration block and the Receiver_Array block. The online debugging modes reveals that the configuration was applied as mentioned. The sender block itself reflects that the string "hello ionia" is received.



Remark:

Please remember before compiling and running this example, the Module_Nr has to be adjusted according your hardware. How to determine the Module_Nr is described in IEC61131 API, d-006935-nnnnnn1.

5.9 WTB Gateway Application: D521_WTB_GW_App_001

The WTB Gateway Application enables the ionia system to actively take part on the communication at train bus level, according to TCN/IEC 61375. Since the TCN standard does not define an application profile, it is up to the application programmer to implement an application profile according its needs. Predefined application profile such as the UIC-556 can be implemented.

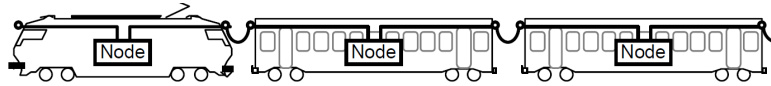


Figure 1 – Wire Train Bus

Figure 1 illustrates the general idea of the WTB application architecture: A WTB node communicate to its consist network and to its partner nodes on the WTB.

5.9.1 Communication Concept

The layout of the Gateway Application Sample D521_WTB_GW_App_001 is ready to communicate with two WTB nodes and a partner device on the MVB consist network.

Of course, more devices could be attached to the consist network as well as to the WTB network. The limiting factor of using two nodes only is the process data mapping implemented in the Gateway Application Sample itself. The process data mapping can be reprogrammed easily by an IEC61131 application programmer.

After the WTB Inauguration, the process data communication over WTB is established, whereas the receiving resp. transmitting data is send resp. read from the corresponding MVB port.

The notation of SRC and Sink port is always with respect to the attached media, i.e MVB resp. WTB.

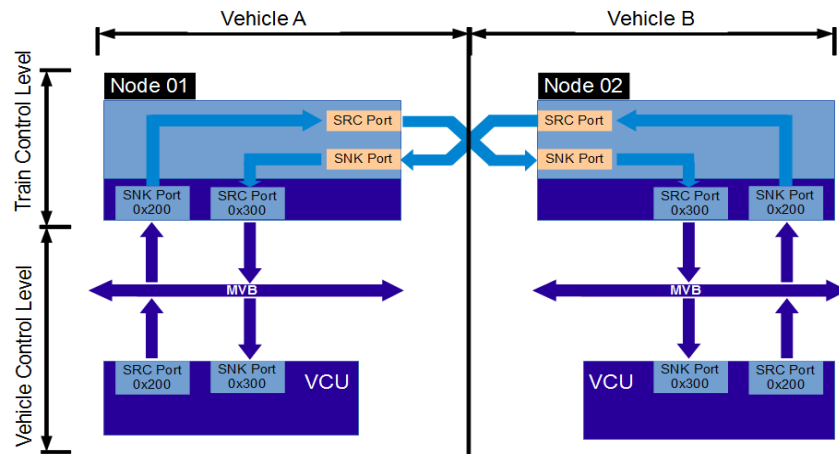
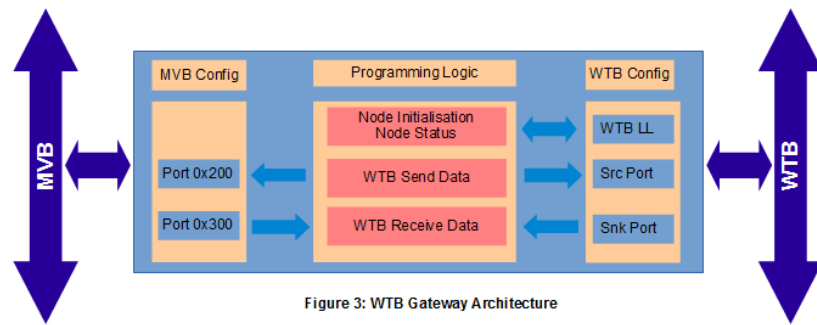


Figure 2: Communication Concept

The WTB Gateway Architecture (Figure 3) shows the programming building blocks of a WTB Nodes. All these parts can be reprogrammed resp. reconfigured according to application specific needs.



In the following are the default settings of the configurable and programmable building blocks described (see figure 3).

5.9.2 MVB Configuration

The design of the MVB configuration is done by means of the duagon D2000 tool. A MVB description file serves as input of the command line tool D2000. This text file is parsed by D2000 and generates among others an appropriate bin file as output. i.e. ionia_01.bin.

The ionia_01.bin file is located in the MULTIPROG project directory:

%MP_PROJE%\projects\D521_WTB-GW_App_001\C\Configuration\R\ionia_01\

The according process variables can be found in MULTIPROG in the Global_Variables worksheet.

Port 0x200

Name	pd_tx
Size	32 Byte
Type	Sink Port

Port 0x200 receives the process data from the consist network and forwards them to the WTB.

Since on the receiver side (Port 0x300) the GW application writes the node strength into byte 0, data byte 0 should not be used otherwise the data will be lost.

The Process variables of Port 0x200 can be found in the Global_Variables worksheet of MULTIPROG (see below).

Name	Type	Usage	Description	Address
⊕ Type_NodeDescriptor				
⊕ Type_Node_Status				
⊖ End Type_WTB_Status				
⊕ Type_Topography				
⊖ End Type_Topography				
⊕ Various TCN Types				
⊕ Service				
⊕ Link Layer Configuration				
⊕ PD Communication				
⊖ MVB COMMUNICATION				
⊖ Port 0x200 PD WTB Send (Sink)				
MVB_WTB_Send_data_byte_000	USINT	VAR_GLOBAL		%IB0512000
MVB_WTB_Send_data_byte_001	USINT	VAR_GLOBAL		%IB0512001
MVB_WTB_Send_data_byte_002	USINT	VAR_GLOBAL		%IB0512002
MVB_WTB_Send_data_byte_003	USINT	VAR_GLOBAL		%IB0512003

The addressing is as described in the chapter 5.5.1 i101_MVB_GetStarted / i301_MVB_GetStarted in Global Variables

Port 0x300

Name	pd_rx
Size	32 Byte
Type	Source Port

The process data received from the other node over the WTB are sent to the consist network by means of Port 0x300.

The first data byte of Port 0x300 contains the strength of the node:

- 0x01 Strong Master
- 0x02 Weak Master
- 0x03 Regular Slave

The Process variables of Port 0x300 can be found in the Global_Variables worksheet of MULTIPROG (see below).

Name	Type	Usage	Description	Address
☐ System Variables				
☐ Link Layer Service L_RESULT				
☐ Report				
☐ Type_WTBStatus				
☐ Link Layer Service State				
☐ End Link Layer Service State				
☐ Type_NodeDescriptor				
☐ Type_Node_Status				
☐ End Type_WTB_Status				
☐ Type_Topography				
☐ End Type_Topography				
☐ Various TCN Types				
☐ Service				
☐ Link Layer Configuration				
☐ PD Communication				
☐ MVB COMMUNICATION				
☐ Port 0x200 PD WTB Send (Sink)				
☐ Port 0x300 PD WTB Receive (Source)				
WTB_Receive_data_byte_000	USINT	VAR_GLOBAL		%QB768000
WTB_Receive_data_byte_001	USINT	VAR_GLOBAL		%QB768001
WTB_Receive_data_byte_002	USINT	VAR_GLOBAL		%QB768002
WTB_Receive_data_byte_003	USINT	VAR_GLOBAL		%QB768003

The addressing is as described in Fehler: Referenz nicht gefunden Fehler: Referenz nicht gefunden.

5.9.3 Programming Logic

The programming logic serves four purposes:

- Node Initialisation.
- Read out node status.
- Prepare the process data to be sent to the WTB.
- Post processing of the received process data from the WTB.

All these tasks are programmed within the IEC 61131 programming environment MULTIPROG Express and are explained in more detail below.

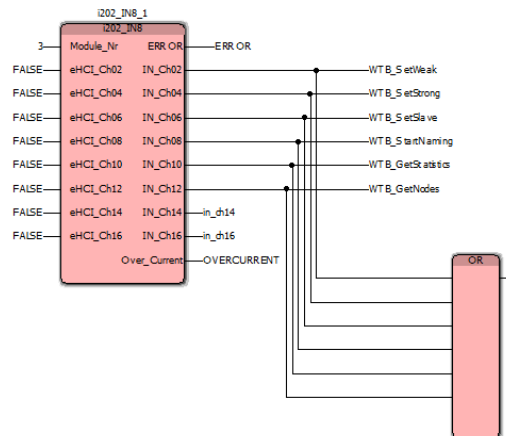
Node Initialisation and Status

The node initialisation is contained in the **Programming Organisation Unit (POU)** **WTB_Main** as Sequential Flow Chart (SFC) program. The initialisation routine configures the WTB Link Layer of the i306 interface card and starts the TCN Inauguration.

The link layer configuration data and the WTB inauguration data are prepared in the **POU WTB_Data**. Where the link layer node descriptor values should remain unchanged, the inauguration data may be changed according to the application specific needs. A successful TCN inauguration returns the WTB_Topography object.

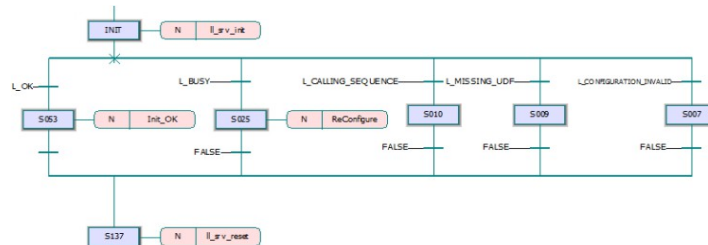
The WTB_Topography Object is read out in the Global_Variables worksheet (see 5.9.4 WTB Configuration).

After a successful TCN inauguration the SFC polls for the WTB Status and WTB Status change requests. The WTB Status Change request commands are mapped to the binary input card i202 channel as shown below:



Thus the WTB status of a node can be changed by means of switches or push buttons connected to module i202. This enables the operator to change the node strength on behalf if required according to the needs of the train composition.

During initialisation as well as when calling WTB status change requests, the return value is evaluated.



In case of a failure, the SFC stops in the appropriate node. A certain application specific action can be performed in this case, e.g. in case of a busy signal, the command can be repeated (see figure below).

WTB Send Data

The process data sent over the WTB is prepared in the **POU WTB_Data**. For this purpose a specific data type for the WTB send data is declared in the MULTIPROG Data_Types declaration work sheet, the struct TYPE_Tx_PD (see below):

```
TYPE
  PD_Array          : ARRAY [0..127] OF USINT;

  TYPE_Tx_PD :
  STRUCT
    node : USINT;
    size : USINT;
    data : PD_Array;
  END_STRUCT;
END_TYPE
```

- **node:** Node number of the sending node. (Own node number.)
- **size:** Size of the sent data in bytes.

- **data:** A data buffer of 128 bytes.

Remark:

The information such as node and size are used to address the WTB Driver accordingly. Only the data bytes are sent over WTB all other data is removed by means of the WTB driver. **The structure of Type_Tx_PD must not be changed.**

WTB Receive Data

The process data received over the WTB is prepared in the **POU WTB_Data**. For this purpose a specific data type for the WTB receive data is declared in the MULTIPROG Data_Types declaration work sheet, the struct TYPE_Rx_PD (see below):

```
TYPE
    PD_Array          : ARRAY [0..127] OF USINT;

    Rx_data :
    STRUCT
        node      : USINT;
        size      : USINT;
        freshness : INT;
        data      : PD_Array;
    END_STRUCT;
    Rx_data_array : ARRAY[0..31] OF Rx_data;

    Type_Rx_PD :
    STRUCT
        nr_nodes : USINT;
        data      : Rx_data_array;
    END_STRUCT;
END_TYPE
```

Rx_data:

- **node:** Node number of the sending node. (Other node number:)
- **size:** Size of the received data in bytes.
- **freshness:** Freshness of the received data.
- **data:** A data buffer of 128 bytes.

Type_Rx_data:

- **nr_nodes:** Number of nodes from which data has been received.
- **data:** A data buffer containing receive data for 31 nodes.

Remark:

Only the data bytes are received from the WTB, whereas the other data is added to the data type by means of the WTB driver, in order to give information to the application programmer. **The structure of Type_Rx_PD must not be changed.**

5.9.4 WTB Configuration

The WTB Link Layer services can be called by means of activating one of the following signals in the **Global Variables** worksheet in the section Link Layer Configuration.

Link Layer Configuration					
ll_cg_transmission_rate	UINT	VAR_GLOBAL	transmission rate in kbit/s, default 1000	%QW20000100	1000
ll_cg_basic_period	UINT	VAR_GLOBAL	basic period in milliseconds, default 25	%QW20000102	25
ll_cg_frilling_disabled	UINT	VAR_GLOBAL	=1 if frilling disabled, default 0	%QW20000104	0
ll_cg_node_descriptor	Type_NodeDescriptor	VAR_GLOBAL	Type_NodeDescriptor	%QD20000106	
ll_cg_poll_md_when_idle	USINT	VAR_GLOBAL	=1 if background scanning enabled	%QB20000110	0
ll_cg_sink_port_count	UINT	VAR_GLOBAL	Maximum number of sink ports, ...	%QW20000111	32
ll_cg_source_port_count	UINT	VAR_GLOBAL	Maximum number of source ports, ...	%QW20000113	1
ll_cg_port_size	USINT	VAR_GLOBAL	Maximum length of a Port in octets, ...	%QB20000115	128
ll_cg_max_number_nodes	USINT	VAR_GLOBAL	Maximum numbers of nodes with ...	%QB20000124	32
ll_cg_inaug_data_max_size	USINT	VAR_GLOBAL	Maximum numbers of octets of ...	%QB20000125	124
ll_cg_inaug_data_size	USINT	VAR_GLOBAL	Actual number of octets of applic...	%QB20000126	4
ll_cg_inaug_data_list	inaug_data_array	VAR_GLOBAL	Pointer to data area where to co...	%QD20000127	

The used addresses are fixed numbers and must not be changed, otherwise a different service is called.

The returned link layer service state is read out in the Global Variables worksheet, in the section Link Layer Service State

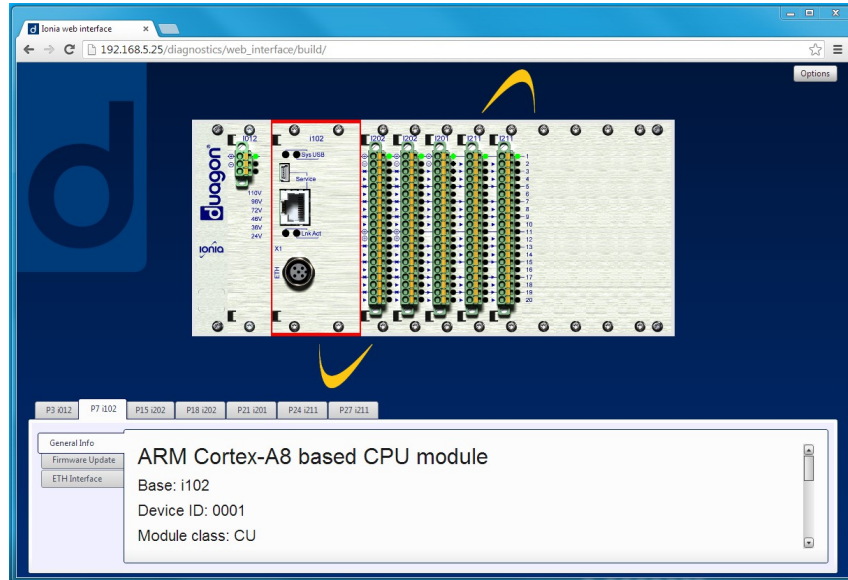
Link Layer Service State				
WTB_STAT_LS_INITIALIZED	BOOL	VAR_GLOBAL	node in state UNCONFIGURED	%MX 3.2000010...
WTB_STAT_LS_CONFIGURED	BOOL	VAR_GLOBAL	node in state CONFIGURED	%MX 3.2000010...
WTB_STAT_LS_READY_TO_NAME	BOOL	VAR_GLOBAL	node in state NAMING_MASTER	%MX 3.2000010...
WTB_STAT_LS_READY_TO_BE_NAMED	BOOL	VAR_GLOBAL	node in state UNNAMED_MAST...	%MX 3.2000010...
WTB_STAT_LS_INHIBIT	BOOL	VAR_GLOBAL	node in Inauguration inhibited	%MX 3.2000010...
WTB_STAT_LS_REGUALR_STRONG	BOOL	VAR_GLOBAL	node in state REGULAR_MAST...	%MX 3.2000010...
WTB_STAT_LS_REGUALR_SLAVE	BOOL	VAR_GLOBAL	node in state REGULAR_SLAVE...	%MX 3.2000010...
WTB_STAT_LS_REGUALR_WEAK	BOOL	VAR_GLOBAL	node in state REGULAR_MAST...	%MX 3.2000010...

With the same method all the TCN Types can be read out and are displayed in the Global Variables work sheet. If complex data types are received than the corresponding structs are expanded in the POU WTB_Data. In the example below, the WTB_TOPO_inauguration_data is expanded as mentioned above.

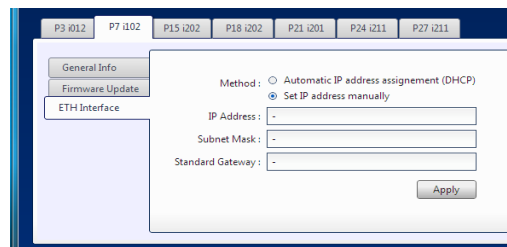
Type_Topography				
WTB_TOPO_node_address	USINT	VAR_GLOBAL		%MB 3.20000200
WTB_TOPO_node_orient	USINT	VAR_GLOBAL		%MB 3.20000201
WTB_TOPO_topo_counter	USINT	VAR_GLOBAL		%MB 3.20000202
WTB_TOPO_individual_period	USINT	VAR_GLOBAL		%MB 3.20000203
WTB_TOPO_is_strong	USINT	VAR_GLOBAL		%MB 3.20000204
WTB_TOPO_number_of_nodes	USINT	VAR_GLOBAL		%MB 3.20000205
WTB_TOPO_bottom_address	USINT	VAR_GLOBAL		%MB 3.20000206
WTB_TOPO_top_address	USINT	VAR_GLOBAL		%MB 3.20000207
WTB_TOPO_inauguration_data	Type_Inauguration_Data	VAR_GLOBAL		%MB 3.20000208

6. IP configuration

The Ethernet interfaces can be configured via the web interface. Start by entering the present IP address of the service Ethernet interface into the web browser and then click on the CPU module:

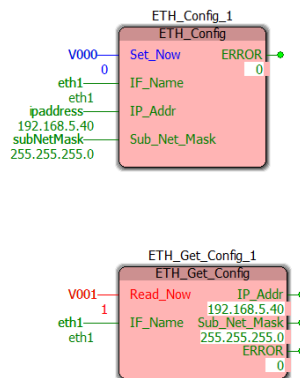


To change the configuration of the service interface (eth0), click on the ETH Interface tab. This opens a dialog where the configuration settings can be entered:



6.0.1 IP configuration from within an IEC61131 program

The ionia library i102_DG_ETH_LIB contains function blocks to get and set the IP address and subnet mask of an Ethernet interface.



For example to configure eth1

Using an ETH_Config FB:

- set IF_Name to 'eth1',
- IP_Addr to the required IP address (e. g. '192.168.5.40')
- and Sub_Net_Mask to the required subnet mask value (e.g. '255.255.255.0').
- As soon as Set_Now is set to TRUE, the settings will be transferred to the ionia, which takes down the appropriate Ethernet interface and brings it back up with the new configuration.

To check the configuration use an ETH_Get_Config FB and:

- set IF_Name to 'eth1'.
- After setting the Read_Now input to TRUE
- the current configuration values appear at the appropriate outputs.

Both FB's use rising edge triggers to synchronise access to the ionia.

7. References

MULTIPROG Express Installer, d-007271-016712

Download link:

http://support.duagon.com/ionia/MULTIPROG_Express_d-007271-016712.exe

Necessary to install the MULTIPROG Express PLC programming tool together with all duagon libraries

ionia i012, i014 short data sheet, d-006282-nnnnnn¹

Short form data sheet of the ionia i012 and i014 power supply modules.

ionia i101, i102, i103 short data sheet, d-006296-nnnnnn¹

Short form data sheet of the ionia i101, i102, i103 CPU modules.

ionia i201/i202 short data sheet, d-006321-nnnnnn¹

Short form data sheet of the ionia i201, i202 digital I/O modules

ionia i205 short data sheet, d-006871-nnnnnn¹

Short form data sheet of the ionia i205 relays module

MVB System User's Guide, d-000656-nnnnnn¹

Introduction and guide to the Multifunction Vehicle Bus (MVB)

IEC61131 API, d-006935-nnnnnn¹

Data sheet of the ionia Multiprog Library elements. These elements are used to access duagon ionia modules.

D2000 MVB Configuration Tool data sheet, d-000341-nnnnnn¹

Data sheet of the D2000 MVB Configuration Tool.

D2000 MVB Configuration Tool, d-000336-nnnnnn¹

D2000 MVB Configuration Tool, is used to build MVB network configurations. The output of a parsed Network configuration file can be downloaded to the ionia TCMS system.

¹ The number "nnnnnn" refers to the latest available revision of this document.

8. Document history

d-007724-014426

New document.

Installer version d-007271-014411

d-007724-015008

Added Ethernet sample application.

Installer version d-007271-014994

d-007724-015105

Added Ethernet configuration.

Added WTB Section.

Installer version d-007271-0nnnnn

d-007724-016184

Sample application replaced trough GetStarted Application for i101, i102, i103, 301, 302, i303, i305

Installer version d-007271-016712

Appendix A: Document Numbering System

All duagon documents have a unique identification number. The identification number has a certain internal structure in order to ease the tracking of different documents. In general, there are two parts:

Prefix	Document number	Filing number
d	-000310	-001952
Always constant	Specifies a certain purpose of a document with the intention to link several documents with different filing number. Please note, that the purpose of the document number is not stored for each document number, but can be derived from the document title, which is stored for each Filing number. The format is either 6 digits or not available.	Unique number, that identifies a particular document. Released in sequential manner as the documents are filed in the archive. A duagon- internal data base contains exactly one document title text for each filing number. Always 6 digits.

Examples for identification numbers

Identification number	Document Title / Remarks
d-000310-001606	„DXIO data sheet Rev 2.2“
d-000310-001952	„DXIO data sheet Rev 2.3“ A document, that is updated from time to time: the document number has the purpose to link several versions of the „DXIO data sheet“ together. The filing number distinguishes between different versions. Please note, that the document number part is kept the same, as long as the basic intention of the early versions is still kept, for example during revisions due to debugging or manufacturing updates. In case a significant change happens, another document number would be applied.
d-000719	„Notes from prototype meeting ...“ A document, that is obviously not updated after release. The „document number“ part is missing and the filing number remains the only used part for identification.

Recommendation:

In your order, you may specify for example "d-000584-nnnnnn" in order to get the "newest" version of a specific product. When you do not want to follow the sequence of newer versions, i.e. you want to stick to a specific version, then specify the full identification number like "d-000584-002043".